# Task 1

```
Query  Query History
1  ALTER TABLE passengers
2  ADD CONSTRAINT check_passenger_age
3  CHECK (date_of_birth <= CURRENT_DATE - INTERVAL '10 years');

Data Output  Messages  Notifications
ALTER TABLE

Query returned successfully in 91 msec.
```

## Explanation:

In this task, I added a CHECK constraint to the passengers table using ALTER TABLE. The constraint ensures that all passengers are at least 10 years old by checking their date of birth. If the age requirement is not met, PostgreSQL rejects the record.

# Task 2

```
5  ALTER TABLE booking
6  ADD CONSTRAINT check_booking_price
7  CHECK (price BETWEEN 0 AND 50000);

Data Output  Messages  Notifications
ALTER TABLE

Query returned successfully in 70 msec.
```

## Explanation:

In this task, I added a CHECK constraint to the booking table using ALTER TABLE. The constraint ensures that ticket prices are between 0 and 50,000 tenge. PostgreSQL rejects any new or updated record with a price outside this range.

# Task 3

```
9   ALTER TABLE baggage
10  ADD CONSTRAINT check_baggage_weight
11  CHECK (weight_in_kg BETWEEN 1 AND 23)
12  NOT VALID;
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 70 msec.
```

## Explanation:

In this task, I added a CHECK constraint to the baggage table using ALTER TABLE. The constraint ensures that baggage weight must be between 1 and 23 kilograms. NOT VALID allows the constraint to be added without checking existing records, while enforcing the rule for new and updated data.

# Task 4

```
14  ALTER TABLE airport
15  ADD CONSTRAINT check_airport_name
16  CHECK (LENGTH(airport_name) >= 10);
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 88 msec.
```

## Explanation:

In this task, I added a CHECK constraint to the airport table using ALTER TABLE. The constraint ensures that airport names contain at least 10 characters. PostgreSQL rejects new or updated records if the airport name is shorter than the required length.

# Task 5

```sql
ALTER TABLE airline
ADD CONSTRAINT unique_airline_name
UNIQUE (airline_name);

ALTER TABLE airport
ADD CONSTRAINT unique_airport_name
UNIQUE (airport_name);

ALTER TABLE passengers
ADD CONSTRAINT unique_passport
UNIQUE (passport_number);

ALTER TABLE flights
ADD CONSTRAINT unique_flight
UNIQUE (flight_no, scheduled_departure, airline_id, departure_airport_id);

ALTER TABLE booking
ADD CONSTRAINT unique_booking
UNIQUE (passenger_id, booking_platform, created_at);
```

```sql
ALTER TABLE baggage
ADD CONSTRAINT unique_baggage
UNIQUE (booking_id, weight_in_kg, created_date);

ALTER TABLE baggage_check
ADD CONSTRAINT unique_baggage_check
UNIQUE (booking_id, passenger_id, created_at);

ALTER TABLE boarding_pass
ADD CONSTRAINT unique_boarding_pass
UNIQUE (booking_id, seat, boarding_time);

ALTER TABLE booking_flight
ADD CONSTRAINT unique_booking_flight
UNIQUE (booking_id, flight_id, created_at);

ALTER TABLE security_check
ADD CONSTRAINT unique_security_check
UNIQUE (passenger_id, created_at, update_at);
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 209 msec.
```

## Explanation:

In this task, I added UNIQUE constraints to all 10 tables using ALTER TABLE. These constraints prevent duplicate values in selected columns or combinations of columns. This helps maintain data integrity and consistency in the database.

# Task 6

```
58  ALTER TABLE passengers
59  ADD CONSTRAINT check_gender_age
60  CHECK (
61      (gender = 'Male' AND date_of_birth <= CURRENT_DATE - INTERVAL '18 years')
62      OR
63      (gender = 'Female' AND date_of_birth <= CURRENT_DATE - INTERVAL '19 years')
64  )
65  NOT VALID;
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 83 msec.
```

## Explanation:

In this task, I added a CHECK constraint to the passengers table based on gender. Male passengers must be at least 18 years old, while female passengers must be at least 19 years old. NOT VALID allows the constraint to be added without checking existing records, while enforcing it for new and updated data.

# Task 7

```
67  ALTER TABLE passengers
68  ADD CONSTRAINT check_citizenship_age
69  CHECK (
70      (country_of_citizenship = 'Kazakhstan'
71       AND date_of_birth <= CURRENT_DATE - INTERVAL '18 years')
72      OR
73      (country_of_citizenship = 'France'
74       AND date_of_birth <= CURRENT_DATE - INTERVAL '17 years')
75      OR
76      (country_of_citizenship NOT IN ('Kazakhstan', 'France')
77       AND date_of_birth <= CURRENT_DATE - INTERVAL '19 years')
78  )
79  NOT VALID;
```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 79 msec.
```

# Explanation:

In this task, I added a CHECK constraint to the passengers table based on citizenship. Passengers from Kazakhstan must be at least 18 years old,

passengers from France at least 17, and passengers from other countries at least 19. NOT VALID allows the constraint to be added without checking existing records.

# Task 8

```
81  ALTER TABLE booking
82  ADD COLUMN ticket_discount DECIMAL(7,2);
83
84  UPDATE booking
85  SET ticket_discount =
86      CASE
87          WHEN created_at >= '2024-01-01'
88          THEN ROUND(price * 0.05, 2)
89          ELSE ROUND(price * 0.10, 2)
90      END;
91
92  ALTER TABLE booking
93  ADD CONSTRAINT check_ticket_discount
```

```
94  CHECK (
95      (created_at >= '2024-01-01'
96       AND ticket_discount = ROUND(price * 0.05, 2))
97      OR
98      (created_at < '2024-01-01'
99       AND ticket_discount = ROUND(price * 0.10, 2))
100 );
101
102 SELECT booking_id, price, created_at, ticket_discount
103 FROM booking
104 LIMIT 10;
```

Data Output | Messages | Notifications

SQL | Showing rows

| | booking_id [PK] integer | price numeric (7,2) | created_at date | ticket_discount numeric (7,2) |
|---|---|---|---|---|
| 1 | 1 | 7462.13 | 2023-06-06 | 746.21 |
| 2 | 2 | 4216.60 | 2024-01-03 | 210.83 |
| 3 | 3 | 5782.37 | 2024-01-12 | 289.12 |
| 4 | 4 | 102.96 | 2024-01-31 | 5.15 |
| 5 | 5 | 6711.04 | 2023-04-25 | 671.10 |
| 6 | 6 | 7813.55 | 2023-12-23 | 781.36 |
| 7 | 7 | 1346.71 | 2024-02-28 | 67.34 |
| 8 | 8 | 5711.67 | 2023-10-06 | 571.17 |
| 9 | 9 | 7015.82 | 2023-12-28 | 701.58 |
| 10 | 10 | 1668.38 | 2024-01-07 | 83.42 |

## Explanation:

In this task, I added a ticket_discount column to the booking table. The discount is calculated based on the ticket price and creation date. Tickets created from January 1, 2024 receive a 5% discount, while earlier tickets receive 10%. A CHECK constraint ensures that the discount values follow these rules.

# Task 9

```
106  ALTER TABLE booking
107  ALTER COLUMN status SET DEFAULT 'Confirmed';
108
109  INSERT INTO booking (
110      booking_id,
111      passenger_id,
112      booking_platform,
113      created_at,
114      update_at,
115      price,
116      ticket_discount
117  )
```

```
117  )
118  SELECT
119      (SELECT MAX(booking_id) + 1 FROM booking),
120      1,
121      'Lab 5 Demo',
122      CURRENT_DATE,
123      CURRENT_DATE,
124      10000,
125      CASE
126          WHEN CURRENT_DATE >= '2024-01-01'
127          THEN 500
128          ELSE 1000
129      END
130  RETURNING booking_id, booking_platform, status, price;
```

Data Output | Messages | Notifications

SQL | Showing r

| | booking_id [PK] integer | booking_platform character varying (50) | status character varying (50) | price numeric (7,2) |
|---|---|---|---|---|
| 1 | 501 | Lab 5 Demo | Confirmed | 10000.00 |

## Explanation:

In this task, I set the default value of the status column to 'Confirmed' using ALTER TABLE. Then I inserted a new booking without specifying its status. PostgreSQL automatically assigned 'Confirmed' to the new record.

# Task 10

```sql
ALTER TABLE airline
ALTER COLUMN created_at SET DEFAULT CURRENT_DATE;

INSERT INTO airline (
    airline_id,
    airline_code,
    airline_name,
    airline_country,
    update_at
)
VALUES (
    (SELECT MAX(airline_id) + 1 FROM airline),
    'LAB5',
    'Lab Five Airways',
    'Kazakhstan',
    CURRENT_DATE
);

SELECT airline_id, airline_name, created_at
FROM airline
WHERE airline_code = 'LAB5';
```

Data Output | Messages | Notifications

Showing

| | airline_id [PK] integer | airline_name character varying (50) | created_at date |
|---|---|---|---|
| 1 | 51 | Lab Five Airways | 2026-10-13 |

## Explanation:

In this task, I used ALTER TABLE to set CURRENT_DATE as the default value for the created_at column. When a new airline is inserted without specifying its creation date, PostgreSQL automatically assigns the current date. This helps ensure consistent date recording.